

# Chapter 3

## Convolutional Neural Networks

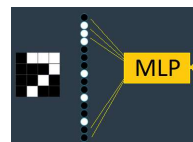
Deep Learning

Dr. Charles Yaacoub

1

### Images with MLP

- Digital Image
  - Matrix: rows and columns
  - Each point: pixel
  - Grayscale: pixel is a scalar value (8-bit image: 0 is black, 255 is white)
  - Color: pixel is a vector
    - Different color spaces; most common: RGB (red, green, blue)
    - Pixel vector has 3 components.
    - In 24-bit RGB, each channel (R, G, and B) is an 8-bit value.
  - Unsigned integers (no negative values) or float/double (normalized to the range [0-1])
- Images and Multilayer Perceptrons
  - MLPs receive input as vector
  - Flattening: convert the image (matrix) to vector
  - Examples: Handwritten Digit Recognition Using PyTorch



<https://towardsdatascience.com/handwritten-digit-mnist-pytorch-977b5338e627>  
[https://github.com/udacity/deep-learning-v2-pytorch/blob/master/convolutional-neural-networks/mnist-mlp/mnist\\_mlp\\_solution\\_with\\_validation.ipynb](https://github.com/udacity/deep-learning-v2-pytorch/blob/master/convolutional-neural-networks/mnist-mlp/mnist_mlp_solution_with_validation.ipynb)

2

Dr. Charles Yaacoub

2

# Digital Images & Computer Vision

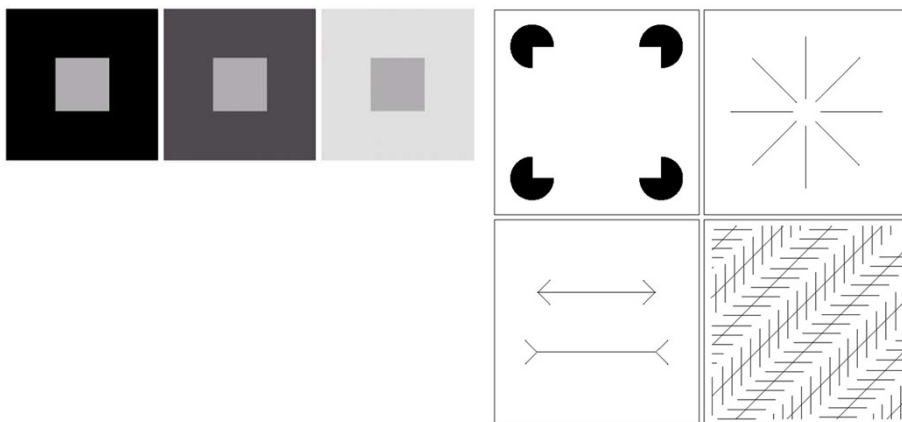
a brief introduction...

Dr. Charles Yaacoub

3

## Vision vs. Computer Vision

- The human vision system is made to “**understand**” the scene, not just to measure values of light
- Illusions



4

Dr. Charles Yaacoub

4

## Vision vs. Computer Vision

- Computers (or machines) can “**see**” (cameras, sensors, ...), but... Can they “**understand**” the scene like humans?
  - Digital Image Processing
  - Attempts to make computers understand what they “see”:  
Computer Vision
    - Where are the cars?
    - Where are humans?
    - How far is that tree?
  - Algorithms for smoothing (noise reduction), sharpening, edge detection, feature extraction, ...
  - Applications: face detection, facial expression recognition, ...
  - Remarkable advances with Artificial Intelligence

5

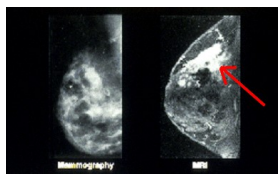
Dr. Charles Yaacoub

5

## Computer Vision: WHY?



Safety



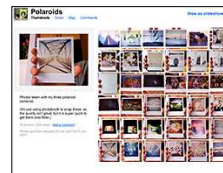
Health



Security



Fun



Access

from James Hays, Brown University

6

Dr. Charles Yaacoub

6

## Some Examples



License plate readers  
[http://en.wikipedia.org/wiki/Automatic\\_number\\_plate\\_recognition](http://en.wikipedia.org/wiki/Automatic_number_plate_recognition)

Many new digital cameras and smart phones now detect faces



from James Hays, Brown University

7

Dr. Charles Yaacoub

7

## Some Examples



Fingerprint scanners on many new laptops, other devices



Face recognition systems now beginning to appear more widely  
<http://www.sensiblevision.com/>

from James Hays, Brown University

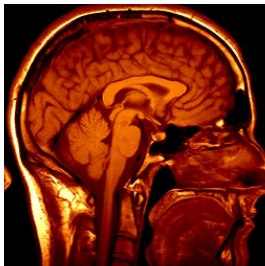
8

Dr. Charles Yaacoub

8

## Some Examples

- Vision in space
- Industrial robots
- Medical imaging
  - Computer aided diagnosis
  - Image guided surgeries
- etc...



3D imaging  
MRI, CT



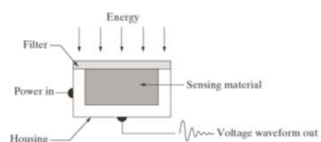
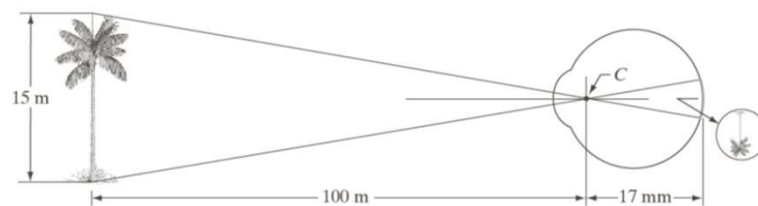
Image guided surgery  
Grimson et al., MIT

9

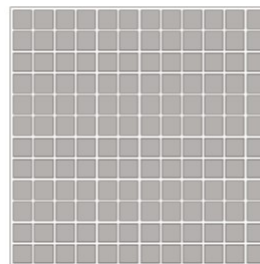
Dr. Charles Yaacoub

9

## Digital Images



Single images sensor vs. line sensor vs. array sensor

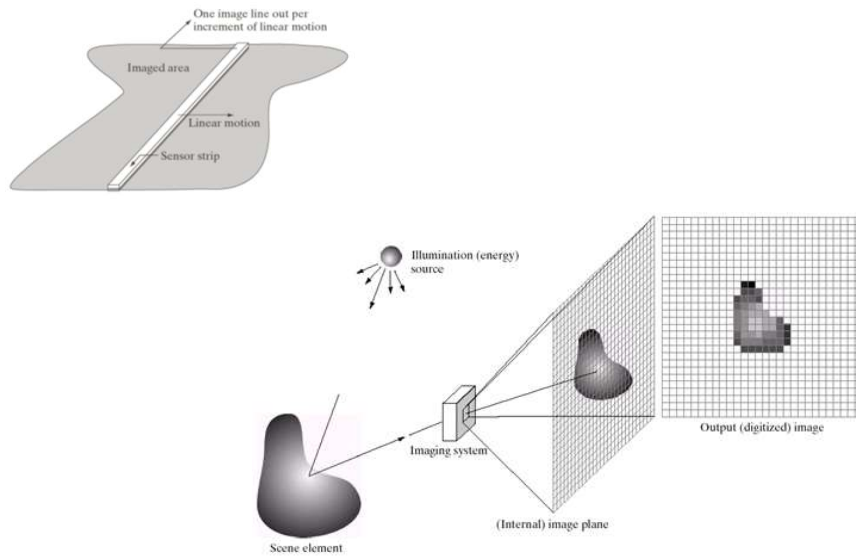


10

Dr. Charles Yaacoub

10

## Digital Images

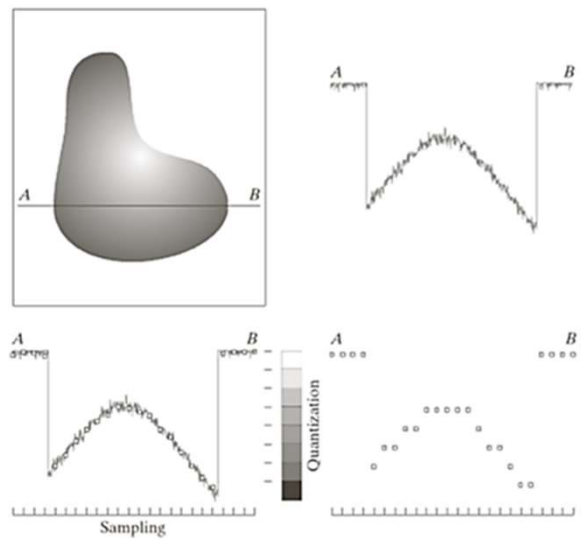


11

Dr. Charles Yaacoub

11

## Digital Images

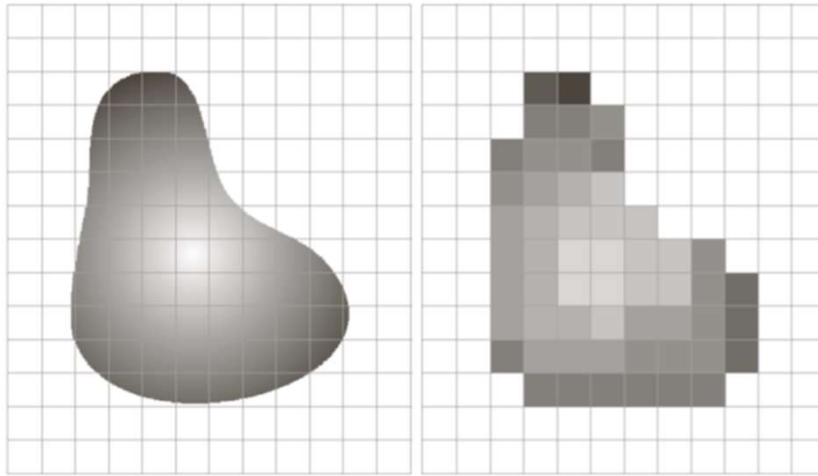


12

Dr. Charles Yaacoub

12

## Digital Images



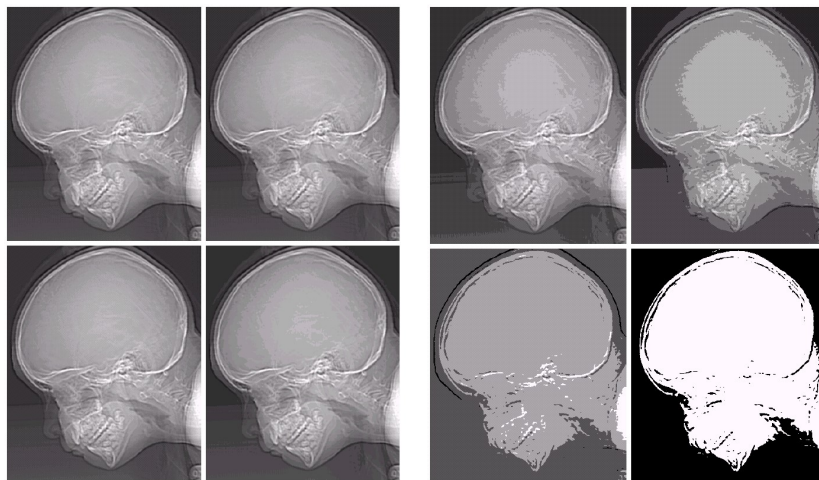
13

Dr. Charles Yaacoub

13

## Digital Images

- Bitdepth: 8, 7, 6, ..., 2, 1 bit(s) per pixel



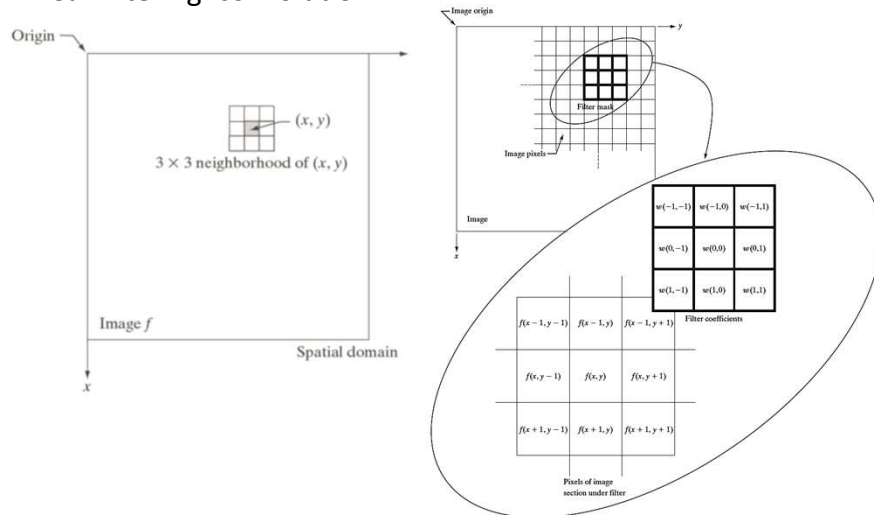
14

Dr. Charles Yaacoub

14

## Image Filters

- Linear filtering: convolution



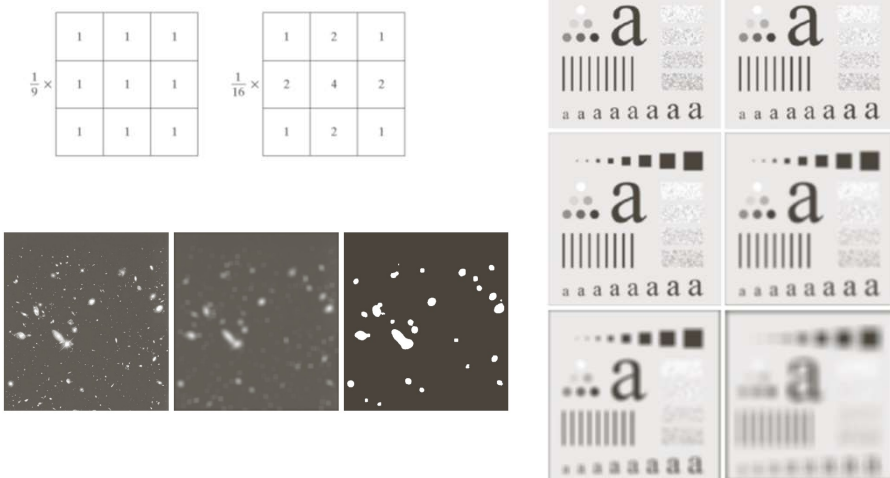
15

Dr. Charles Yaacoub

15

## Image Filters

- Smoothing examples



16

Dr. Charles Yaacoub

16

## Image Filters

- Edge detection and sharpening example

|    |    |    |    |    |    |
|----|----|----|----|----|----|
| 0  | 1  | 0  | 1  | 1  | 1  |
| 1  | -4 | 1  | 1  | -8 | 1  |
| 0  | 1  | 0  | 1  | 1  | 1  |
| 0  | -1 | 0  | -1 | -1 | -1 |
| -1 | 4  | -1 | -1 | 8  | -1 |
| 0  | -1 | 0  | -1 | -1 | -1 |



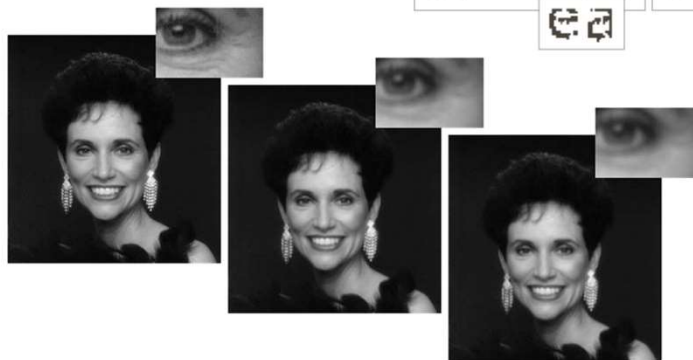
17

Dr. Charles Yaacoub

17

## Image Filters

- Applications



Historically, certain computer programs were written using only two digits rather than four to define the applicable year. Accordingly, the company's software may recognize a date using "00" as 1900 rather than the year 2000.



Historically, certain computer programs were written using only two digits rather than four to define the applicable year. Accordingly, the company's software may recognize a date using "00" as 1900 rather than the year 2000.

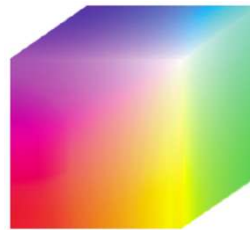
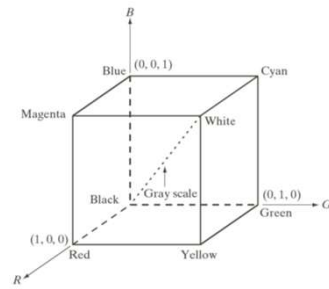
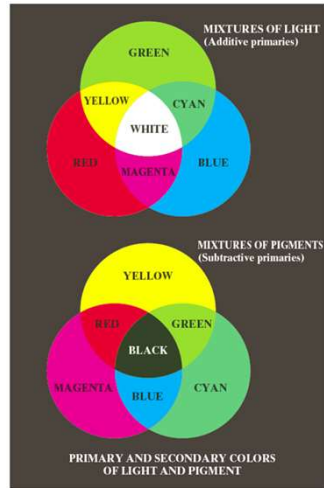


18

Dr. Charles Yaacoub

18

## Color Images

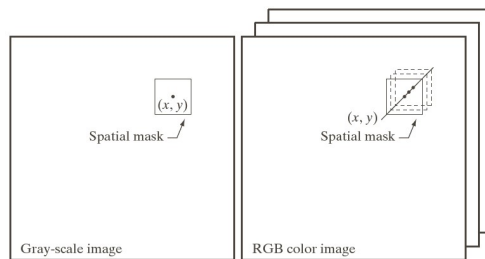
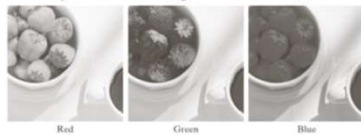
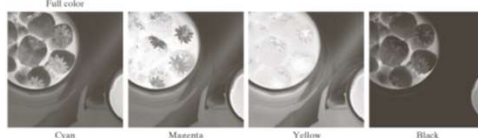


19

Dr. Charles Yaacoub

19

## Color Images



20

Dr. Charles Yaacoub

20

## Computer Vision with OpenCV

- Selected notebooks:

01\_Getting\_Started\_with\_Images

02\_Basic\_Image\_Manipulation

03\_Image\_Annotation

04\_Image\_Enhancement

06\_Video\_Writing

07\_Image\_Filtering\_Edge\_Detection

08\_Image\_Fetaures\_and\_Alignment

12\_Face\_Detection

13\_Object\_Detection

- All notebooks (OpenCV python Course): <https://opencv.org/opencv-free-course/>
- Drawing on images with a mouse : <https://www.geeksforgeeks.org/drawing-with-mouse-on-images-using-python-opencv/>

21

Dr. Charles Yaacoub

21

## Chapter 3 Convolutional Neural Networks

Deep Learning

Dr. Charles Yaacoub

22

## Convolutional Neural Networks (CNNs)

- CNNs perform better with images, compared to MLPs
- CNNs v/s MLPs
  - CNNs can accept matrices (2D, images) as input  
MLPs only accept vectors
  - MLPs only have fully connected layers, i.e. each node at some layer is connected to every other node in the next layer  
CNNs can have sparsely connected layers, this allows for analyzing different image regions (details later)
- Convolutional Layers
  - Perform **convolution** between an image (input or previous layer) and filter **kernels**
  - **Kernel**: small matrix (usually smaller than the image) containing filter coefficients (different kernels for smoothing, sharpening, ...)

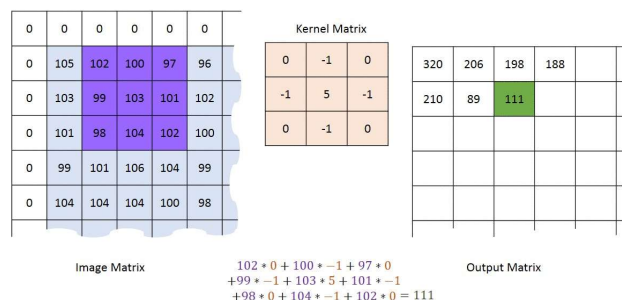
23

Dr. Charles Yaacoub

23

## Convolutional Neural Networks (CNNs)

- Convolution
  - Image  $i(x, y)$ , kernel (or filter)  $k(x, y)$ , result  $g(x, y)$ :  
 $g(x, y) = i(x, y) * k(x, y)$ ;  
 usually equivalent to correlation (symmetric kernels):  
 for all  $(x, y)$ :  $g(x, y) = \sum_m \sum_n i(x+m, y+n) k(m, n)$
  - For same output size: **pad** input with zeros, or duplicate image borders



24

Dr. Charles Yaacoub

24

## Convolutional Neural Networks (CNNs)

- **Stride:** amount (nbr of pixels) by which we move the kernel while performing convolution. (previous example: stride = 1)
- Convolutional layer:
  - Convolve current layer (input image, or set of images = output of previous layer) with set of kernels
  - Each kernel results in a new image fed to the next layer
  - Result of each convolution: **feature map**
  - Stride S:
    - S=1: output image size = input image size
    - S=2: output image size = input image size / 2
    - In general: output image size = input image size / S (in each dimension)

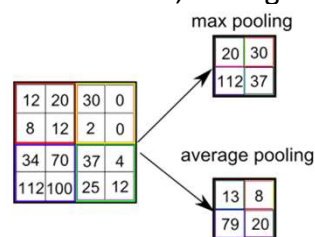
25

Dr. Charles Yaacoub

25

## Convolutional Neural Networks (CNNs)

- Pooling:
  - Reduce the dimensionality
    - With a large number of kernels, output grows larger
    - Pooling: subsampling the output
  - Pool size m x n:
    - Take 1 value from each m x n window (usually square windows m=n)
    - Different pooling types; most common: Max, Average
    - Examples: 2x2 pooling with a stride of 2



26

Dr. Charles Yaacoub

26

## Convolutional Neural Networks (CNNs)

- General Architecture:
  - Sequence of:
    - Convolutional layer
    - Activation (e.g. ReLU)
    - Pooling (e.g. max, avg, ...)
  - Flattening the output (converting to a 1D vector)
  - Fully connected layers (as in MLPs)
- It can also include:
  - Normalization layers: usually placed after activations, to obtain features with a zero mean and a unit variance (this reduces the bias due to means and variances, and reduces the number of epochs required for training the model)
  - Dropout layers

27

Dr. Charles Yaacoub

27

## Convolutional Neural Networks (CNNs)

- Example:

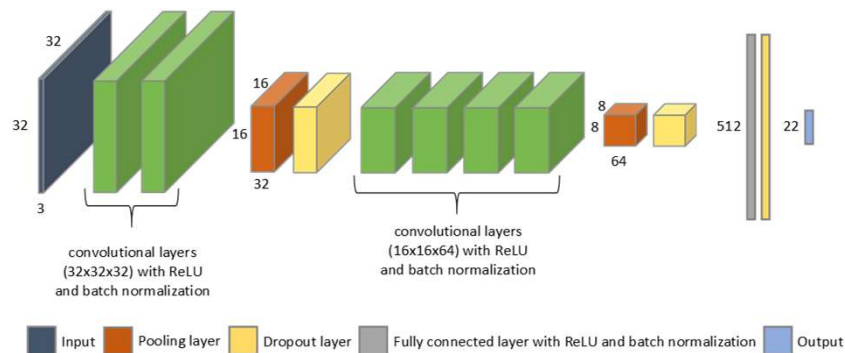


Image from: R. Fermanian, C. Yaacoub, A. Akl and P. Bilane, "A Deep Convolutional Neural Network for Character Recognition in Ancient Syriac Manuscripts," 2022 IEEE 24th International Workshop on Multimedia Signal Processing (MMSP), Shanghai, China, 2022, pp. 1-5, doi: [10.1109/MMSP55362.2022.9949229](https://doi.org/10.1109/MMSP55362.2022.9949229)

28

Dr. Charles Yaacoub

28

## Convolutional Neural Networks (CNNs)

- Defining a convolutional layer in Pytorch:  
**self.conv1 = nn.Conv2d(in\_channels, out\_channels, kernel\_size, stride=1, padding=0)**
  - For example, for an input RGB image (in\_channels = 3) and desired 16 feature maps (out\_channels = 16), a kernel of 3x3 (kernel\_size = 3), output dimensions same as input (stride = 1, padding = 1)  
**self.conv1 = nn.Conv2d(3, 16, 3, stride=1, padding=1)**
- Usually the number of kernels increases in the sequence of layers.  
For example:  
self.conv1 = nn.Conv2d(3, 16, 3, stride=1, padding=1)  
self.conv2 = nn.Conv2d(16, 32, 3, stride=1, padding=1)  
self.conv3 = nn.Conv2d(32, 64, 3, stride=1, padding=1)

29

Dr. Charles Yaacoub

29

## Convolutional Neural Networks (CNNs)

- Define a pooling layer:  
(<https://pytorch.org/docs/stable/nn.html#pooling-layers>)  
**self.maxpool = nn.MaxPool2d(kernel\_size, stride)**  
  
>>> # pool of square window of size=2, stride=2  
>>> self.maxpool = nn.MaxPool2d(2, stride=2)  
>>> # pool of non-square window  
>>> self.maxpool = nn.MaxPool2d((3, 2), stride=(2, 1))
- In many cases, especially in classification problems, we would like our output to be independent from the object's position, orientation, or size.
  - Solution: data augmentation

30

Dr. Charles Yaacoub

30

## Convolutional Neural Networks (CNNs)

- Data augmentation: add more images to your dataset
  - Rotated versions of originals: for rotation invariance
  - Shifted versions of originals: for shift (or position) invariance
  - Scaled versions (i.e. smaller or larger) of originals: for scale (size) invariance
  - Etc...
- Built-in functions provided for data augmentation
- **A CNN example in Pytorch with data augmentation**

Download and run the following Notebook from Udacity:

[https://github.com/udacity/deep-learning-v2-pytorch/blob/master/convolutional-neural-networks/cifar-cnn/cifar10\\_cnn\\_solution.ipynb](https://github.com/udacity/deep-learning-v2-pytorch/blob/master/convolutional-neural-networks/cifar-cnn/cifar10_cnn_solution.ipynb)

Carefully read the notebook while running the different cells. Identify code chunks for data augmentation, CNN design, training, testing, etc...

31

Dr. Charles Yaacoub

31

## Convolutional Neural Networks (CNNs)

- Existing models
  - Pytorch provides definitions of existing models
    - Pre-trained on known datasets
    - Each model can be loaded and used in code
  - Pre-trained: no need for training
  - Retrain the model on your custom dataset
  - Transfer learning
    - retrain parts of the network
    - Possibility of modifying or adding layers
    - (details later)
  - List of models used for classification:  
<https://pytorch.org/vision/stable/models.html>

32

Dr. Charles Yaacoub

32

## Transfer Learning

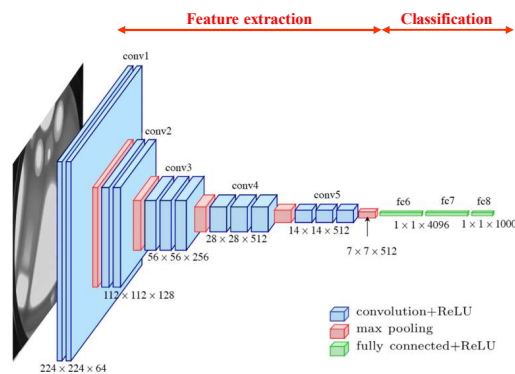
- Why design a new model if you can use an existing one?
  - Existing models are trained on large datasets
  - Complex models with large datasets take considerable time to train
    - Training could take several weeks with multiple GPUs
    - This might not be convenient on low-cost personal computers (no GPU, slow processor, limited RAM and storage, ...)
- We can:
  - Customize existing model and train on new dataset
    - Train only new/modified layers
    - or
    - Retrain the customized model
  - Keep existing model and fine tune the training
  - Use the model architecture and retrain with own dataset

33

Dr. Charles Yaacoub

33

## Transfer Learning



- Example: VGG16 model (figure: model architecture)
- In Pytorch:
 

```
import torchvision
from torchvision import models
# Load the pretrained model
vgg16 = models.vgg16(pretrained=True)
```

34

Dr. Charles Yaacoub

34

## Transfer Learning

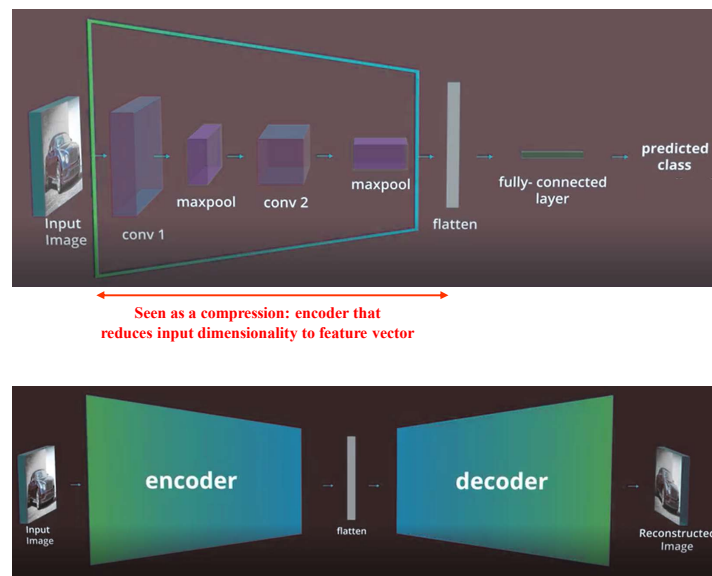
- Recommendations based on type of own dataset:
  - Large dataset
    - Similar to original:  
**Freeze** first few layers (simple features), train remaining layers
    - Different from original:
      - use the network architecture and retrain from scratch
      - randomly initialize weights or start with trained weights
  - Small dataset
    - Similar to original:  
Train only the last fully connected layer
    - Different from original:  
Train only classification layers (fully connected layers), freeze the feature extraction part of the networks

35

Dr. Charles Yaacoub

35

## Autoencoders



36

Dr. Charles Yaacoub

36

## Autoencoders

- Structure: 3 main components
  - Encoder: extracts input features
  - Code: flattened features (convert features to vector)
  - Decoder: given the code, reconstructs the input
- Q: Why use a NN if its role is to reconstruct the input?
- A: Many applications
  - Compression
  - Denoising
  - Image colorization
  - other...

37

Dr. Charles Yaacoub

37

## Autoencoders

- Encoder: reduces dimensionality with convolutions, strides, and pooling.
- Decoder: increases the dimensions of a compressed input to produce the same shape at the output
  - Reverse the downsampling operation
  - Upsampling (un-pooling)
  - Mostly using “Transpose Convolutional Layer”
    - Example: (stride = 1)

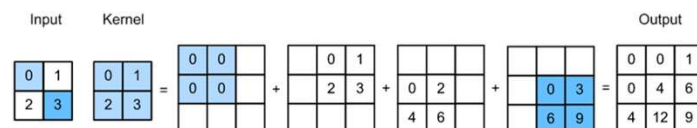


Image source: <https://towardsdatascience.com/transposed-convolution-demystified-84ca81b4baba>

In pytorch:

`nn.ConvTranspose2d(in_channels, out_channels, kernel_size, stride=1)`

38

Dr. Charles Yaacoub

38

# Chapter 3

## Convolutional Neural Networks

Deep Learning

Dr. Charles Yaacoub